Departamento de Engenharia de Electrónica e Telecomunicações e de Computadores

Módulo 1

Autores: 51561 Afonso Jesus
51445 José Saldanha
51447 Ricardo Pinto

Relatório para a Unidade Curricular de Comunicação Digital da Licenciatura em Engenharia Informática e de Computadores

Professor: Artur Ferreira

28 – 04 – 2025

<< Esta página foi intencionalmente deixada em branco >>

Índice

1. Exercícios Introdutórios sobre a linguagem Python 4
   1.1. Função multiplo4(left,right) 4
   1.2. Função mdc(a, b) 4
   1.3. Função pg(N, u, r) 4
   1.4. Função n_combinacoes(n, k) 4
   1.5. Função min_max_vetor(vetor) 5
2. Análise de fontes de símbolos 5
   2.1. Símbolo mais frequente, respetiva probabilidade e informação própia 5
   2.2. Entropia 7
   2.3. Histogramas 8
3. Implementação de fontes de símbolos 13
   3.1. Fonte genérica de símbolos 13
   3.2. Geração de padrões específicos 13
   3.3. Geração de ficheiros com entropia controlada 15
   3.4. Histograma e entropia dos ficheiros anteriores 16
4. Compressão de dados 18
   4.1. Entropia e compressão 18
   4.2. Gráfico Entropia vs Razão de Compressão 19
5. Cifra de ficheiros 20
   5.1. Cifra e Decifra com Vigenère 20
   5.2. Resultados da cifra/decifra 21

1. Exercícios Introdutórios sobre a linguagem Python

1.1. Função multiplo4(left,right)

Descrição: Gera uma lista de múltiplos de 4 no intervalo left a right, inclusive.
Resultados experimentais:

- multiplo4(1, 20) → [4, 8, 12, 16, 20]
- multiplo4(0, 15) → [0, 4, 8, 12]
- multiplo4(5, 5) → []

Comentário: A função funciona corretamente, percorrendo o intervalo left a right e utilizando o operador módulo (%) para verificar a divisibilidade por 4, incluindo os extremos quando são múltiplos de 4. Devolve uma lista vazia quando nenhum múltiplo está presente.

1.2. Função mdc(a, b)

Descrição: Calcula o máximo divisor comum entre dois inteiros, utilizando o algoritmo de Euclides. O algoritmo realiza divisões inteiras sucessivas entre a e b, substituindo a por b e b pelo resto da divisão, até que b seja igual a 0.
Resultados experimentais:

- mdc(48, 18) → 6
- mdc(100, 25) → 25
- mdc(7, 3) → 1

Comentário: Os resultados estão de acordo com os valores esperados segundo o algoritmo de Euclides. A função comporta-se corretamente para diferentes pares de inteiros.

1.3. Função pg(N, u, r)

Descrição: Gera os primeiros *N* termos de uma progressão geométrica com termo inicial *u* e razão *r*.
Resultados experimentais:

- pg(5, 2, 3) → [2, 6, 18, 54, 162]
- pg(4, 1, 2) → [1, 2, 4, 8]
- pg(3, 5, 1) → [5, 5, 5]

Comentário: A função itera *N* vezes, atualizando o valor de *u* multiplicando pela razão *r*. Apresenta um comportamento correto para qualquer valor da razão, incluindo o caso da razão 1 (progressão constante).

1.4. Função n_combinacoes(n, k)

Descrição: Calcula o número de combinações possíveis de k elementos escolhidos de um conjunto de n elementos, utilizando a fórmula de binómio de Newton.

Resultados experimentais:

- n_combinações(5,2) → 10
- n_combinações(6,3) → 20
- n_combinações(10,5) → 252

Comentário: A função usa math.factorial() para implementar diretamente a fórmula:

$$\frac{n!}{k!\,(n-k)!}$$

A verificação "n<k" está presente para evitar valores inválidos.

1.5. Função min_max_vetor(vetor)

Descrição: Percorre o vetor e devolve o menor e o maior valor encontrados. No caso de o vetor estar vazio, devolve "(None, None)".

Resultados experimentais:

- min_max_vetor([3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]) → (1, 9)
- min_max_vetor([0, -1, 1]) → (-1, 1)
- min_max_vetor([7]) → (7, 7)
- min_max_vetor([]) → (None, None)

Comentários: A abordagem implementada garante complexidade linear no tempo de execução, evitando o uso das funções embutidas min() e max() para promover a prática do controlo de fluxos.

2. Análise de fontes de símbolos

Esta secção aborda a análise estatística de ficheiros extraídos do ficheiro *TestFilesCD.zip*. Foram analisados os símbolos presentes em cada ficheiro, a sua frequência, probabilidade e informação. Para além disso, calculou-se a entropia de cada ficheiro, bem como um histograma dos símbolos.

2.1. Símbolo mais frequente, respetiva probabilidade e informação própia

Para determinar o símbolo mais frequente, recorreu-se à classe *Counter* da biblioteca padrão *collections* do *Python*, que permitiu contar a frequência de ocorrência de elementos de cada ficheiro. Como o resultado é devolvido ordenado do mais frequente para o menos frequente, foi possível obter diretamente o símbolo mais comum como o primeiro elemento da lista.

Para calcular a probabilidade do símbolo, utilizou-se simplesmente a frequência desse símbolo, dividida pelo número total de símbolos.

A informação própria de um símbolo foi calculada com base na fórmula definida pela Teoria da Informação:

$$I(x) = -\boldsymbol{log}2\big(p(x)\big)\ [bits]$$

Resultados experimentais:

- a.txt
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.1420
    - Informação Própria: 2.8163 bits
- alice29.txt
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.1900
    - Informação Própria: 2.3957 bits
- arrays.kt
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.3418
    - Informação Própria: 1.5490 bits
- barries.jpg
    - Símbolo mais frequente: ‘\x00‘
    - Probabilidade: 0.0086
    - Informação Própria: 6.8580 bits
- barries.tif
    - Símbolo mais frequente: ‘\xF4‘
    - Probabilidade: 0.0120
    - Informação Própria: 6.3786 bits
- bird.gif
    - Símbolo mais frequente: ‘\x02‘
    - Probabilidade: 0.0093
    - Informação Própria: 6.7515 bits
- cp.htm
    - Símbolo mais frequente: ‘e‘ (Código em ASCII: 101)
    - Probabilidade: 0.0611
    - Informação Própria: 4.0320 bits
- fibonacci.kt
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.2345
    - Informação Própria: 2.0921 bits
- lena.bmp
    - Símbolo mais frequente: ‘\x9B‘

    - Probabilidade: 0.0109
    - Informação Própria: 6.5197 bits
- maximumSubarray.kt
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.2784
    - Informação Própria: 1.8448 bits
- person.java
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.2755
    - Informação Própria: 1.8601 bits
- progc.c
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.1748
    - Informação Própria: 2.5158 bits
- view.kt
    - Símbolo mais frequente: ‘ ‘ (Código em ASCII: 32)
    - Probabilidade: 0.2195
    - Informação Própria: 2.1876 bits

Comentário: Como se pode verificar, nos ficheiros de texto, o símbolo mais comum corresponde ao espaço. Já nos ficheiros de imagem o símbolo mais frequente podes corresponder a um terminador ou uma cor. (Parte da imagem tirei do GPT)

2.2. Entropia

Para além do valor da probabilidade e da informação própria do símbolo mais frequente também foi calculado a valor da entropia do ficheiro. A fórmula utilizada corresponde à definida na Teoria da Informação:

$$H(X) = -\sum p(x_i) * \log_2\big(p(x_i)\big) \ [bits/símbolo]$$

Resultados experimentais:

- a.txt
    - Entropia: 4.8558 bits/símbolo
- alice29.txt
    - Entropia: 4.5677 bits/símbolo
- arrays.kt
    - Entropia: 4.3375 bits/símbolo
- barries.jpg
    - Entropia: 7.9687 bits/símbolo
- barries.tif
    - Entropia: 7.6494 bits/símbolo
- bird.gif

    - Entropia: 7.8944 bits/símbolo
- cp.htm
    - Entropia: 5.2291 bits/símbolo
- fibonacci.kt
    - Entropia: 4.7929 bits/símbolo
- lena.bmp
    - Entropia: 7.4588 bits/símbolo
- maximumSubarray.kt
    - Entropia: 4.4567 bits/símbolo
- person.java
    - Entropia: 4.4591 bits/símbolo
- progc.c
    - Entropia: 5.1989 bits/símbolo
- view.kt
    - Entropia: 5.0205 bits/símbolo

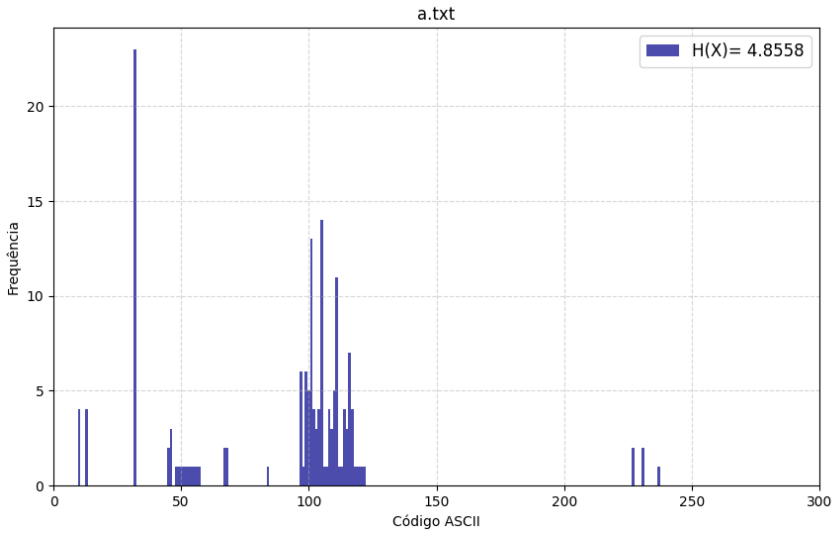
Comentário: Através das entropias observadas, verificou-se que os ficheiros de texto apresentam valores de entropia mais baixos (entre 4 e 5.3) quando comparados com os ficheiros de imagem (entre 7.5 e 7.9). Esta diferença pode ser justificada pelo número de símbolos distintos em cada tipo de ficheiro: ficheiros de texto tendem a utilizar um conjunto mais reduzido de símbolos, ao passo que ficheiros de imagem apresentam uma maior variedade.

2.3. Histogramas

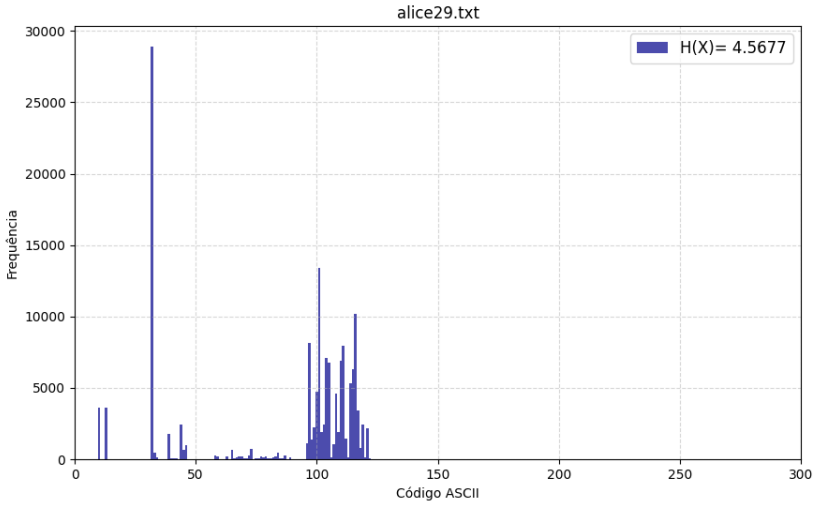
Para os histogramas utilizados neste exercício (e nos seguintes), recorreu-se ao módulo *pyplot* da biblioteca *matplotlib*. Esta ferramenta permitiu visualizar graficamente a frequência de cada símbolo, com base no ser código ASCII.

Resultados experimentais:

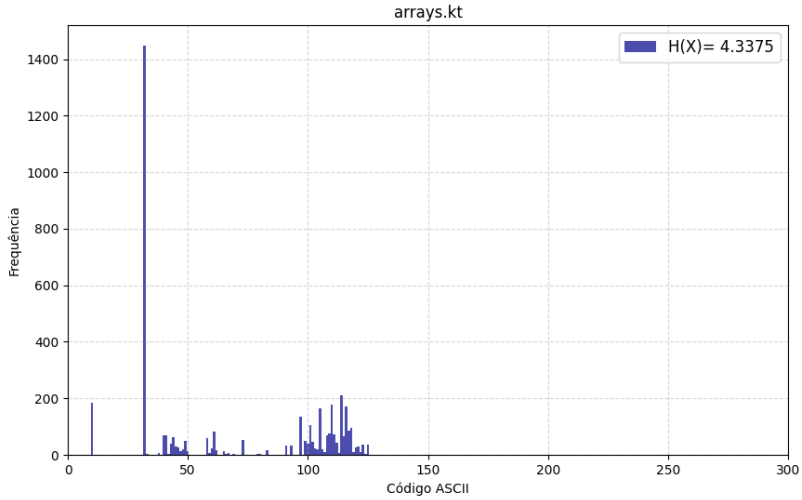
- a.txt



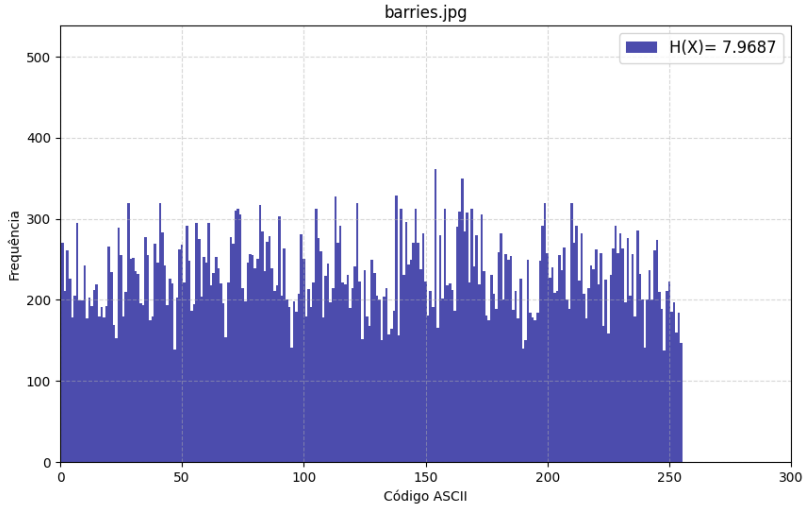
- alice29.txt



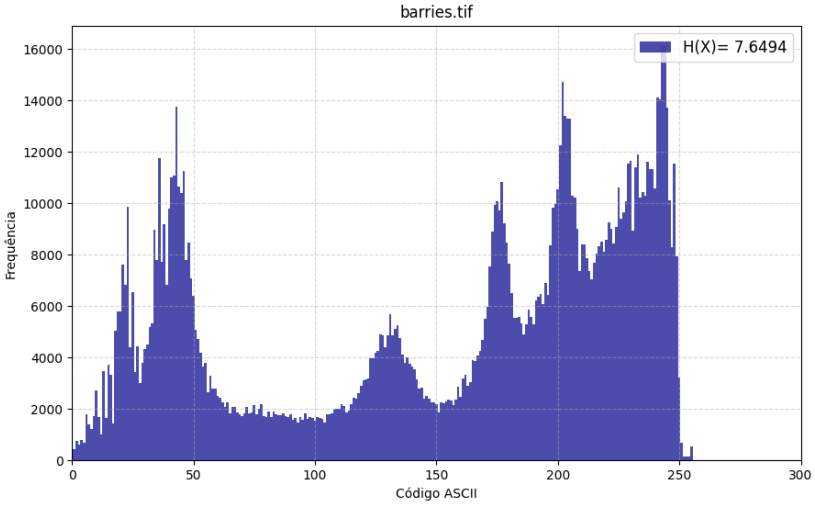
- arrays.kt



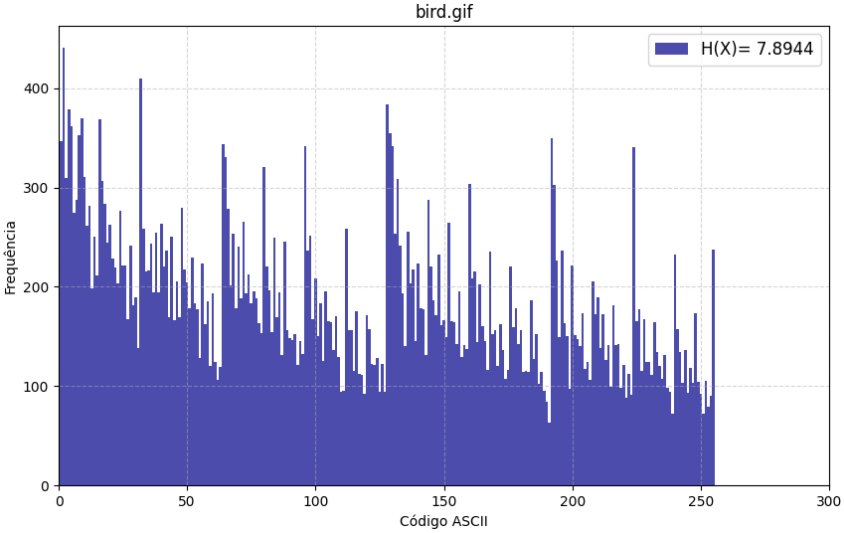
- barries.jpg



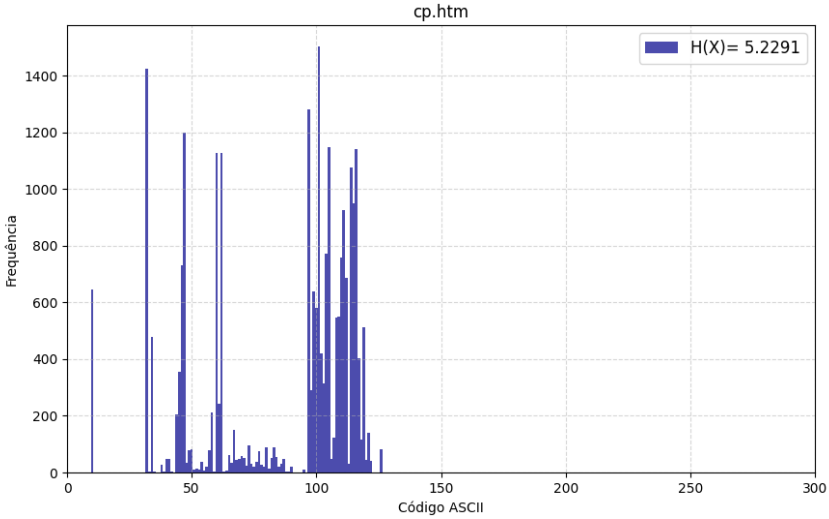
- barries.tif



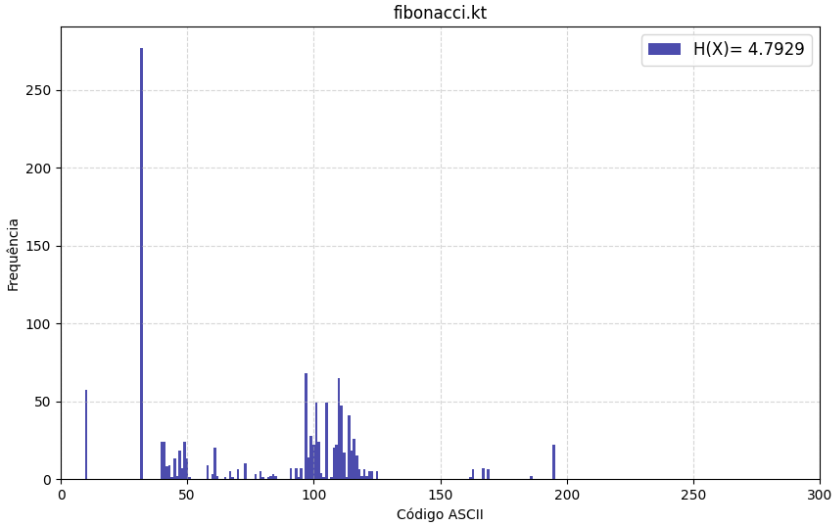
- bird.gif



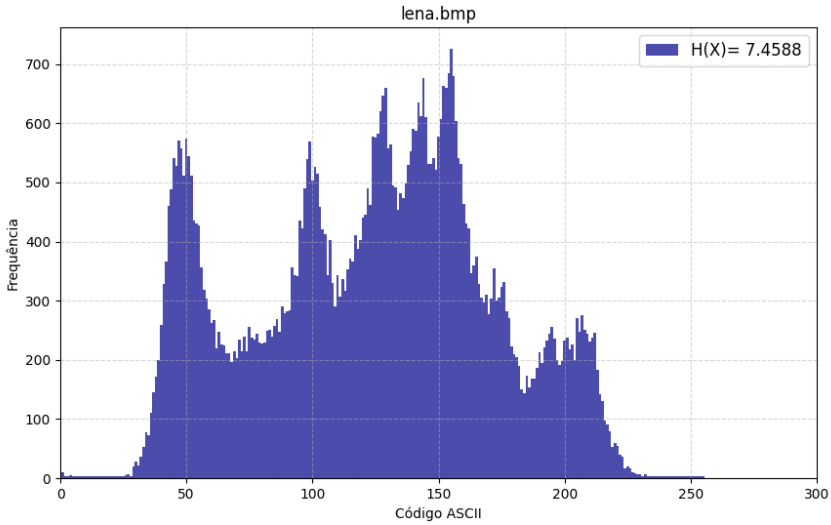
- cp.htm



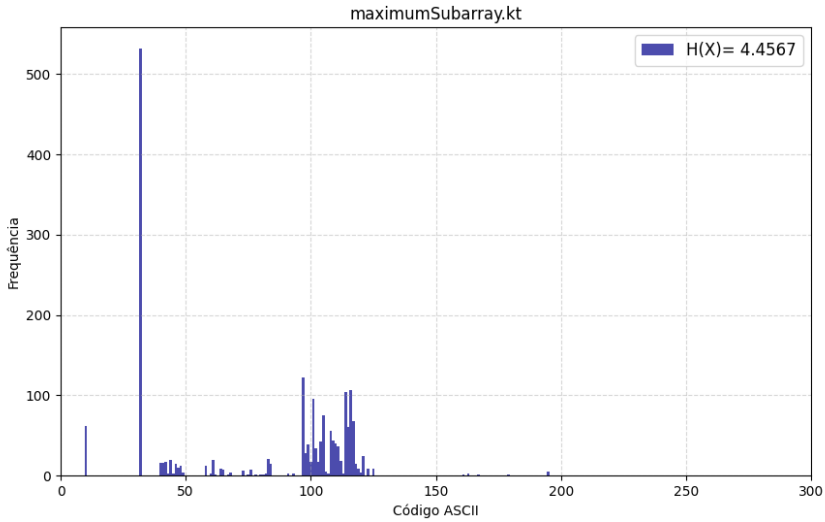
- fibonacci.kt



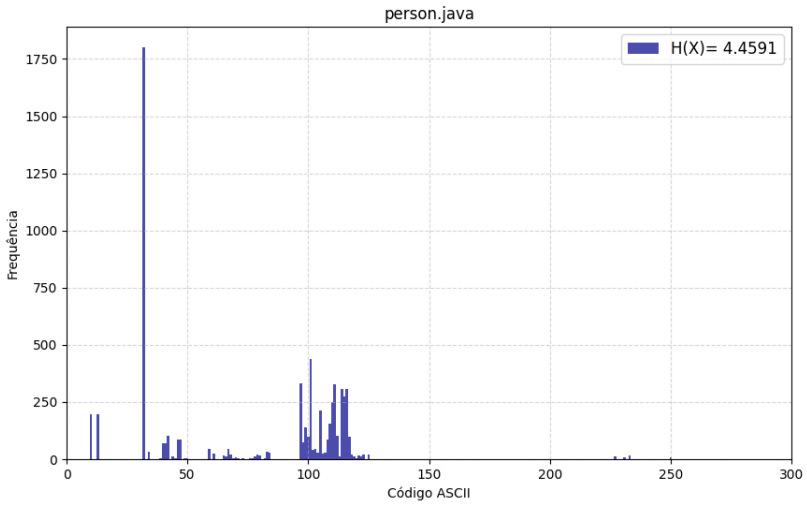
- lena.bmp



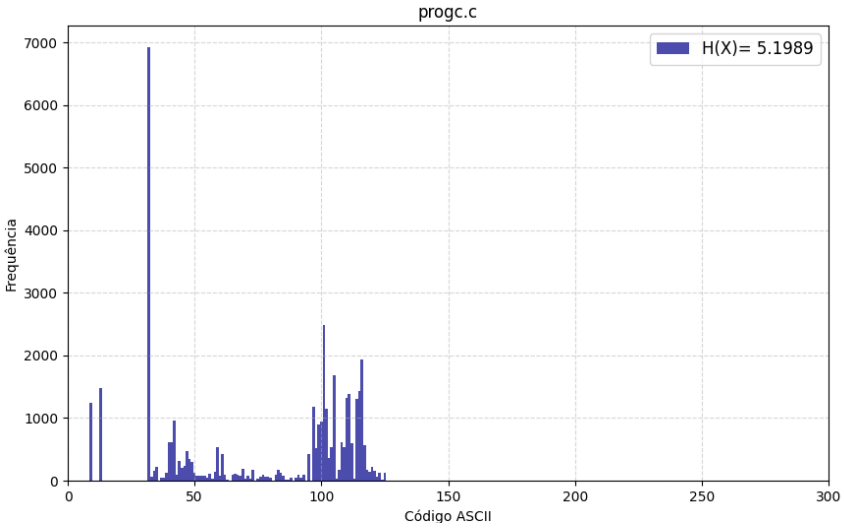
- maximumSubarray.kt



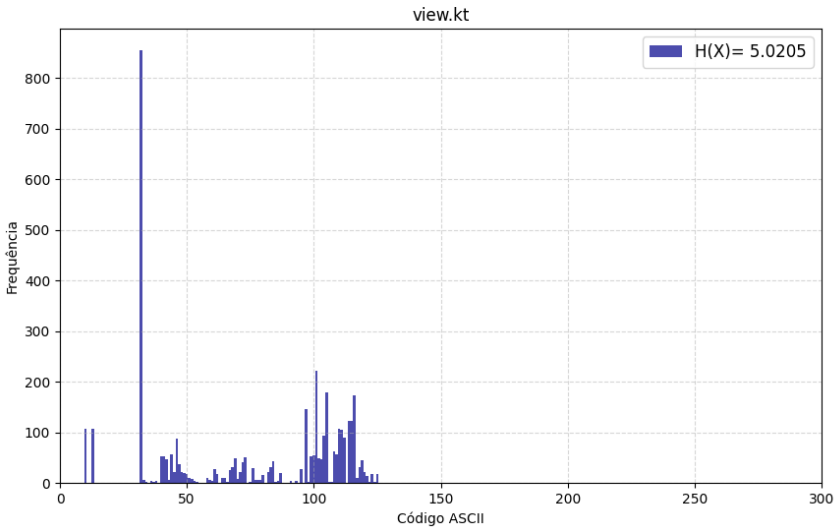
- person.java



- progc.c



- view.kt



3. Implementação de fontes de símbolos

Nesta secção, abordou-se a implementação de fontes de símbolos. Para este exercício, utilizou-se o módulo *random* da biblioteca *numpy* para gerar símbolos aleatórios.

3.1. Fonte genérica de símbolos

Foi implementada uma função que gera sequências de símbolos aleatórios com base numa Função Massa de Probabilidade (FMP) definida. O alfabeto de entrada é composto por M símbolos distintos. A função gera uma sequência de N símbolos, respeitando a FMP definida para o alfabeto.

Inicialmente, para cada símbolo, é calculada a quantidade esperada com base na sua probabilidade e no número total de símbolos a gerar. Em seguida, esses símbolos são adicionados à sequência. Como o arredondamento pode provocar pequenas discrepâncias no comprimento da sequência, é realizada uma correção: se o número de símbolos for inferior a N, adicionam-se símbolos escolhidos aleatoriamente de acordo com a FMP; se for superior, removem-se símbolos em excesso. No final, a sequência é baralhada aleatoriamente para evitar agrupamentos artificiais.

O funcionamento foi validado com um alfabeto de quatro símbolos ‘a’, ‘b’, ‘c’ e ‘d’ e a respetiva FMP (0.8, 0.1, 0.1, 0.0). Os testes demonstraram que a função gera corretamente, respeitando as probabilidades fornecidas. Para garantir isto, utilizou-se a função *choice* do módulo *random* da biblioteca *numpy*, que permite selecionar símbolos com base numa FMP.

3.2. Geração de padrões específicos

Para estes exercícios, conforme solicitado, recorreu-se à fonte genérica de símbolos desenvolvida no exercício anterior. Em cada um dos casos, a Função Massa de Probabilidade (FMP) foi calculada de forma uniforme, garantindo que todos os símbolos do alfabeto definido tinham a mesma probabilidade de ocorrência.

3.2.1. Palavras-passe Robustas

Neste exercício, utilizou-se a função *randint* do módulo random da biblioteca numpy para gerar um número aleatório entre 10 e 14, correspondendo ao comprimento da palavra-passe. Para calcular a FMP de cada carácter, aplicou-se a fórmula $\frac{1}{comprimento(caracteres)}$, garantindo uma distribuição uniforme sobre o alfabeto. O alfabeto é composto por letras (maiúsculas e minúsculas), dígitos e pontuação. Por fim, foi invocada a função *gerar_sequencia*, desenvolvida anteriormente, para criar a sequência de caracteres que compõe a palavra-passe.

Resultados experimentais (exemplo, podem variar consoante corremos o ficheiro):

- ^9<8Vne!q6H(|
- WoHfcm?^.XO_
- |=?s:Q0Qhupu

- N*=4;2o*jsDz|5
- A35'0FN&5G
- z2xf<K%c.<84QZ
- MF-GyYK:S\
- 6^^/\(gjob
- |r*:Y3r`?(@q
- jQr+!2~uxV~kD

3.2.2. Endereços IPv4

Descrição: Para este exercício, definiu-se um alfabeto composto pelas *strings* correspondentes aos números inteiros de 0 a 255. A FMP foi também definida de forma uniforme, com $1/256$ para cada elemento. A sequência gerada é constituída por 4 elementos (octetos), que são posteriormente unidos através do separador ".", formando assim um endereço IPv4 válido. A geração da sequência é igualmente realizada pela função *gerar_sequencia*.

Resultados experimentais (exemplo, podem variar consoante corremos o ficheiro):

- 82.82.233.171
- 109.133.197.121
- 104.218.6.113
- 116.134.218.32
- 241.73.190.228
- 167.16.186.195
- 190.181.19.113
- 13.201.90.22
- 127.109.98.84
- 149.252.100.171

3.2.3. Endereços IPv6

Descrição: Neste caso, seguiu-se uma estrutura semelhante à da geração dos endereços IPv4. No entanto, aqui é necessário gerar 8 grupos de 4 caracteres hexadecimais cada (valores de 0 a 9 e de 'a' a 'f'). O alfabeto foi ajustado para conter apenas esses caracteres, e a FMP foi definida como $1/16$, garantindo distribuição uniforme. Após gerar cada um dos 8 grupos com recurso à função *gerar_sequencia*, os mesmos são concatenados utilizando o separador ":", originando um endereço IPv6.

Resultados experimentais (exemplo, podem variar consoante corremos o ficheiro):

- 9205:59fe:13f3:8ebb:8280:94cf:73d2:fd8f
- 7f02:ee22:14c3:4046:c7c1:b73a:ec33:5642
- 7ebe:164c:bb96:5d15:84ea:f6e3:6fc6:985b
- 31f7:80f8:14fa:f216:d7c3:c181:9459:baee

- 414a:f134:9e5a:d486:1ffe:456a:7eca:52fe
- 3b66:5751:6fa9:f743:6758:eb0f:d0a6:6d63
- 225b:f949:80d4:0e15:4058:d051:a1f4:db4f
- 8e4d:e2cb:2ccb:89d6:8fed:544d:41d7:5397
- 1d97:6e30:ea5c:9997:536e:d677:28e4:51e4
- faeb:1494:87c2:52a3:c7e3:354d:1f5b:6559

3.2.4. Tuplos Hexadecimais

Descrição: Para este exercício, aplicou-se uma abordagem semelhante à usada na geração dos endereços IPv6. No entanto, em vez de criar 8 grupos de 4 caracteres, gerou-se uma única sequência com 8 caracteres hexadecimais (alfabeto de 0 a 9 e 'A' a 'F'). A FMP manteve-se igual, ou seja, $1/16$, assegurando uma distribuição uniforme sobre o alfabeto. A sequência final representa um tuplo hexadecimal.

Resultados experimentais (exemplo, podem variar consoante corremos o ficheiro):

- B76BC8F6
- 0849D469
- 632B06DA
- 0099511A
- AA33ACB2
- 3061D831
- D55997C3
- 63F3DC1D
- D0A045B7
- 614ED428

3.3. Geração de ficheiros com entropia controlada

Para este exercício, decidimos implementar duas funções distintas, cada uma responsável por uma tarefa específica:

- *save_to_file(filename, data)* – guarda os símbolos gerados num ficheiro;
- *generate_file(filename, M, N, H)* – função principal que gera uma sequência de N símbolos com base num alfabeto e probabilidades fixos, e guarda-os no ficheiro indicado.

Definimos manualmente os alfabetos e as distribuições de probabilidade para garantir o cumprimento dos requisitos de entropia estabelecidos. Esta abordagem permitiu-nos um controlo total sobre a geração dos dados.

Para a criação dos dois primeiros ficheiros, selecionámos um alfabeto composto pelos símbolos 'A', 'B', 'C' e 'D', ao qual foi atribuída a Função Massa de Probabilidade (FMP) composta pelos seguintes valores: 0.5, 0.25, 0.125 e 0.125, respetivamente. Esta FMP foi escolhida de forma a assegurar uma entropia próxima de 1.75 bits por símbolo.

Gerámos uma sequência de 100 símbolos para o primeiro ficheiro e de 1000 símbolos para o segundo.

Para o terceiro ficheiro, utilizámos um alfabeto de 256 símbolos, correspondentes aos números inteiros de 0 a 255. Neste caso, apenas os primeiros 16 símbolos têm probabilidade não nula, fixada em 0.0625 para cada um, enquanto os restantes 240 símbolos têm probabilidade nula. Esta configuração resulta numa entropia de 4.0 bits por símbolo.

Todas as sequências geradas foram escritas nos respetivos ficheiros de texto, sendo cada símbolo colocado numa linha distinta. Esta metodologia assegurou a precisão necessária para satisfazer as condições de entropia especificadas no enunciado.

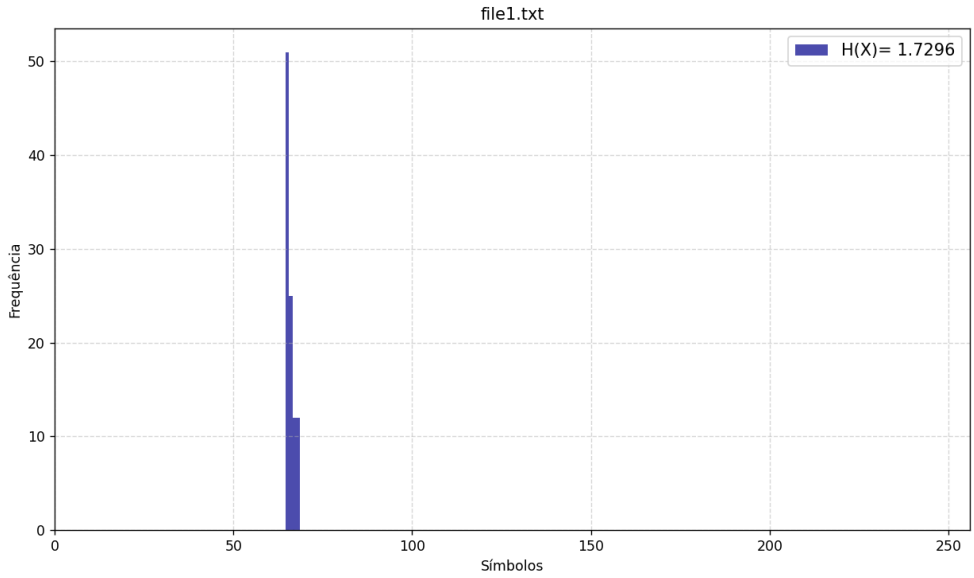
3.4. Histograma e entropia dos ficheiros anteriores

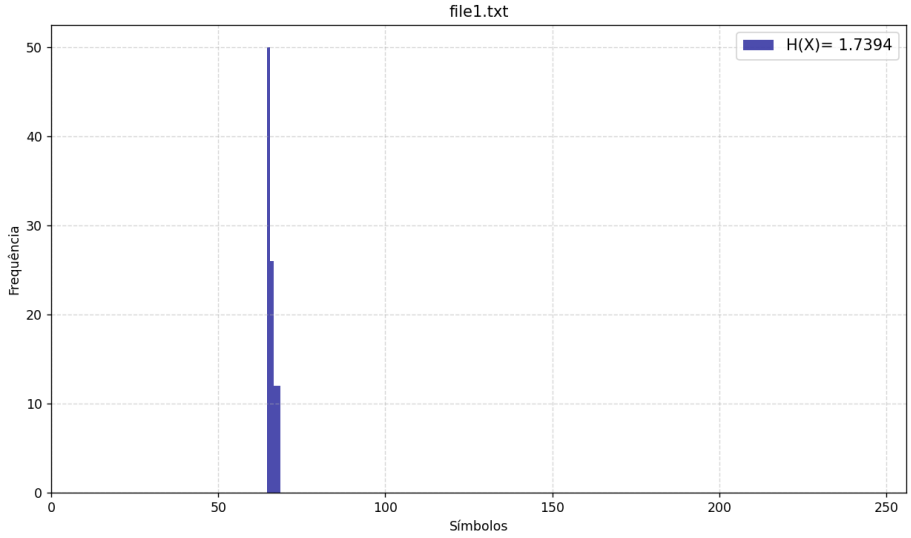
Ao analisarmos a entropia dos ficheiros anteriores, reparámos que:

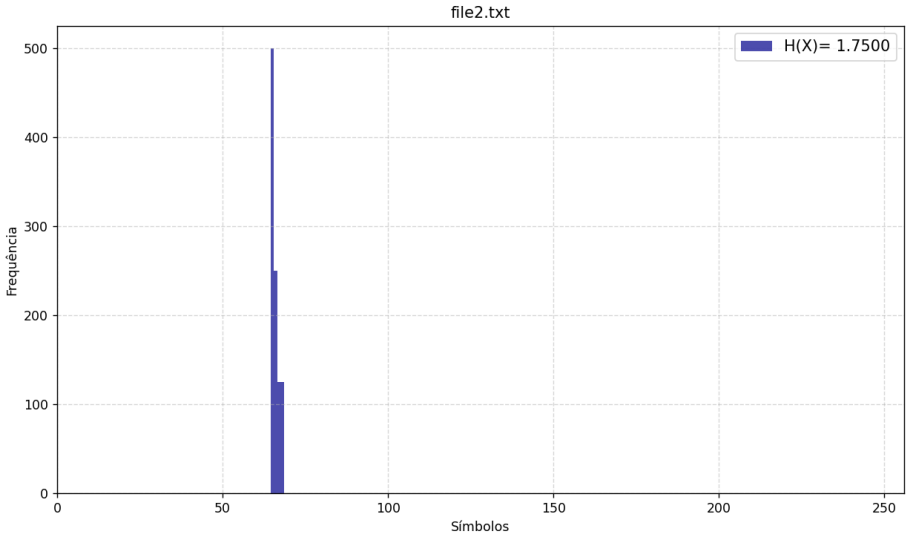
- o 1º ficheiro que pedia uma entropia de 1.75, obtivemos uma entropia entre 1,73 e 1,74 (dependendo da execução), o que representa uma diferença ligeiramente inferior ao valor pretendido, mas ainda aceitável. Esta ligeira discrepância deve-se ao arredondamento das quantidades de símbolos geradas com base nas probabilidades fornecidas. Como o número total de símbolos (N=100) é relativamente pequeno, o arredondamento pode introduzir variações mais significativas na distribuição real dos símbolos;
- o 2º ficheiro que também requeria uma entropia de 1,75, obtivemos exatamente o valor esperado, confirmando a precisão do processo;
- o 3º ficheiro, que exigia uma entropia de 4,00, obtivemos igualmente uma entropia de 4,00, correspondendo exatamente ao valor estipulado.

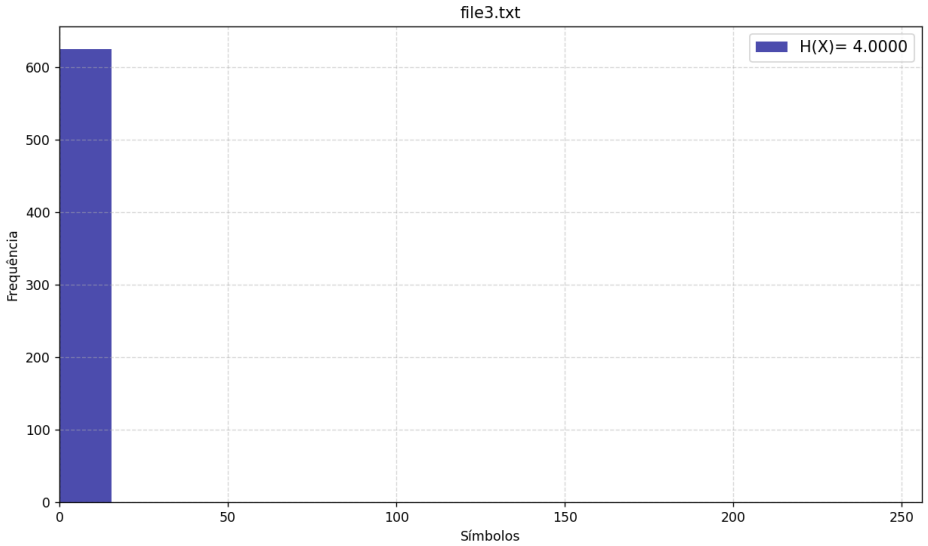
Assim, os resultados demonstram que o método utilizado para gerar a Função de Massa de Probabilidade (FMP) é eficaz, rigoroso e fiável, permitindo controlar com grande precisão a entropia das sequências geradas. As pequenas diferenças observadas podem ser atribuídas a variações estatísticas normais associadas ao processo de geração aleatória.

Histogramas:




file1.txt
H(X)= 1.7394
Frequência
Símbolos
0
10
20
30
40
50
0
50
100
150
200
250


file2.txt
H(X)= 1.7500
Frequência
Símbolos
0
100
200
300
400
500
0
50
100
150
200
250


file3.txt
H(X)= 4.0000
Frequência
Símbolos
0
100
200
300
400
500
600
0
50
100
150
200
250

4. Compressão de dados

4.1. Entropia e compressão

Descrição: Neste exercício, decidimos fazer a compressão com recurso ao módulo *zipfile* da biblioteca do *Python*, que utiliza o algoritmo *DEFLATE*, similar ao utilizado em ferramentas como o 7-Zip ou WinZip.

Assim, inicialmente extraímos os contéudos do ficheiro zip inicial, lemos cada ficheiro, cálculamos a entropia associada a cada ficheiro e por fim comprimimos o ficheiro e cálculamos a razão de compressão com base na fórmula:

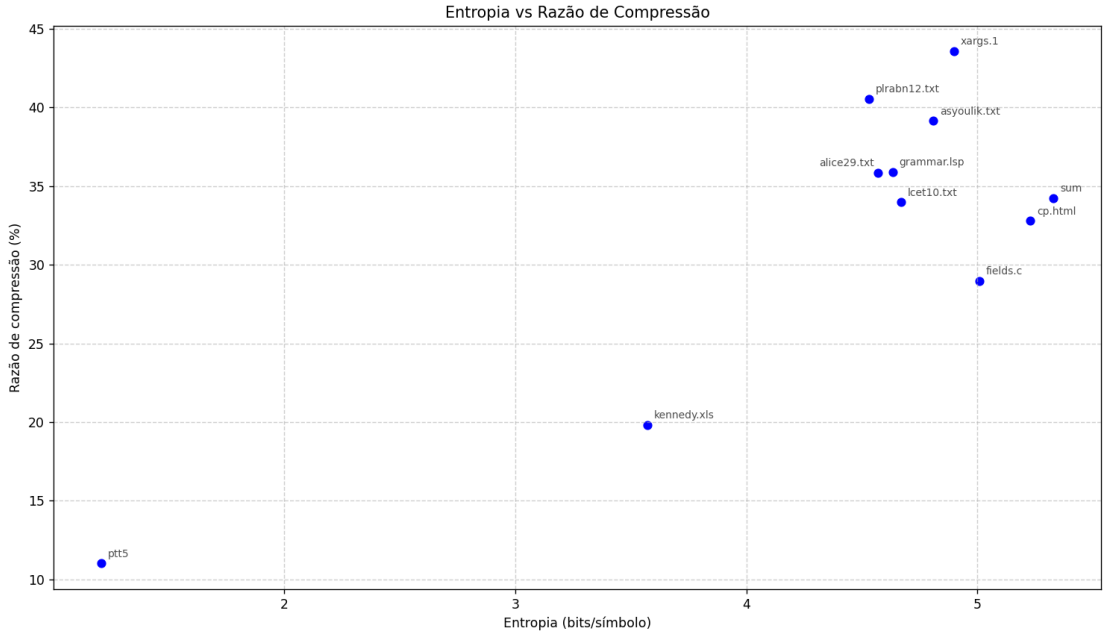
$$\text{Razão de compressão } = \left(\frac{Tamanho\ comprimido}{Tamanho\ original}\right) * 100\ [\%]$$

Resultados experimentais:

- alice29.txt
  - Entropia: 4.5677 bits/símbolo
  - Tamanho original: 152089 bytes
  - Tamanho comprimido: 54518 bytes
  - Razão de Compressão: 0.3585 → 35.85%
- asyoulik.txt
  - Entropia: 4.8081 bits/símbolo
  - Tamanho original: 125179 bytes
  - Tamanho comprimido: 49013 bytes
  - Razão de compressão: 0.3915 → 39.15%
- cp.html
  - Entropia: 5.2291 bits/símbolo
  - Tamanho original: 24603 bytes
  - Tamanho comprimido: 8067 bytes
  - Razão de compressão: 0.3279 → 32.79%
- fields.c
  - Entropia: 5.0077 bits/símbolo
  - Tamanho original: 11150 bytes
  - Tamanho comprimido: 3230 bytes
  - Razão de compressão: 0.2897 → 28.97%
- grammar.lsp
  - Entropia: 4.6323 bits/símbolo
  - Tamanho original: 3721 bytes
  - Tamanho comprimido: 1339 bytes
  - Razão de compressão: 0.3590 → 35.90%
- kennedy.xls
  - Entropia: 3.5735 bits/símbolo
  - Tamanho original: 1029744 bytes

    - Tamanho comprimido: 204106 bytes
    - Razão de compressão: 0.1982 → 19.82%
- lcet10.txt
    - Entropia: 4.6691 bits/símbolo
    - Tamanho original: 426754 bytes
    - Tamanho comprimido: 145016 bytes
    - Razão de compressão: 0.3398 → 33.98%
- plrabn12.txt
    - Entropia: 4.5314 bits/símbolo
    - Tamanho original: 481861 bytes
    - Tamanho comprimido: 195377 bytes
    - Razão de compressão: 0.4055 → 40.55%
- ptt5
    - Entropia: 1.2102 bits/símbolo
    - Tamanho original: 513216 bytes
    - Tamanho comprimido: 56565 bytes
    - Razão de compressão: 0.1102 → 11.02%
- sum
    - Entropia: 5.3290 bits/símbolo
    - Tamanho original: 38240 bytes
    - Tamanho comprimido: 13088 bytes
    - Razão de compressão: 0.3423 → 34.23%
- xargs.1
    - Entropia: 4.8984 bits/símbolo
    - Tamanho original: 4227 bytes
    - Tamanho comprimido: 1842 bytes
    - Razão de compressão: 0.4358 → 43.58%

4.2. Gráfico Entropia vs Razão de Compressão



5. Cifra de ficheiros

5.1. Cifra e Decifra com Vigenère

Neste exercício foi implementada uma versão personalizada da cifra de Viginère, aplicada a ficheiros de texto. A cifre permite cifrar letras, dígitos e espaços, seguindo os seguintes critérios:

- Letras alfabéticas são cifradas de acordo com o tradicional algoritmo de Vigenère, distinguindo entre maiúsculas e minúsculas;
- Dígitos numéricos são igualmente sujeitos a deslocamentos baseados na chave, assegurando um maior grau de ofuscação;
- Espaços são substituídos aleatoriamente por caracteres especiais ('¤', '¢'), com o objetivo de disfarçar a estrutura do texto original;
- Demais caracteres (pontuação, símbolos, etc) são preservados sem alterações

Como é típico neste tipo de cifra, a chave é repetida ciclicamente ao longo do texto.

Desenvolvemos duas funções para aplicar o algoritmo a ficheiros:

- *encrypt_file* – lê o conteúdo de um ficheiro, aplica a cifra e escreve o resultado cifrado noutro ficheiro
- *decrypt_file* – realiza o processo inverso, recuperando o texto original a partir da versão cifrada.

Assim, este método permite aplicar a cifra a qualquer ficheiro de texto plano.

Exemplo:

Queremos decifrar este texto plano com a chave 'KEY': OLA OLA OLA OLA AOGAJSFLASKGBJASGA AZEITE12313. Aplicando o algoritmo para encriptar este texto e recebemos este resultado: YPY#YPY#YPY&YPY%KSEKNQPPYCOELNYCKY&KDCSXC45646. Se agora decidirmos desencriptar o texto usando a função para desencriptar, teremos de usar a mesma chave usada para encriptar, ou seja, temos de utilizar a chave 'KEY', assim, recebemos o seguinte resultado: OLA OLA OLA OLA AOGAJSFLASKGBJASGA AZEITE12313. (O resulta do texto cifrado pode e deve ser usado como um exemplo, que ocorreu devido a uma execução do código que gerou esta cifra)

Comentário: Este exemplo demonstra que a cifra implementada consegue encriptar e desencriptar corretamente o conteúdo de um ficheiro, desde que seja utilizada a mesma chave. Além disso, o uso de substituição dos espaços por símbolos especiais introduz uma camada adicional de disfarce, útil para reduzir padrões visuais que poderiam ser explorados em ataques por análise de frequência.

5.2. Resultados da cifra/decifra

Neste exercício, a cifra e a decifra foram aplicadas aos ficheiros “alice29.txt” e “asyoulik.txt”. Decidimos cifrar e decifrar ambos os ficheiros com a chave ‘KEYS’. Após a cifra e a subsequente decifra, analisámos a entropia dos ficheiros e representámos graficamente os histogramas da frequência dos símbolos, de modo a verificar o comportamento da distribuição dos caracteres ao longo das transformações.

Resultados experimentais:

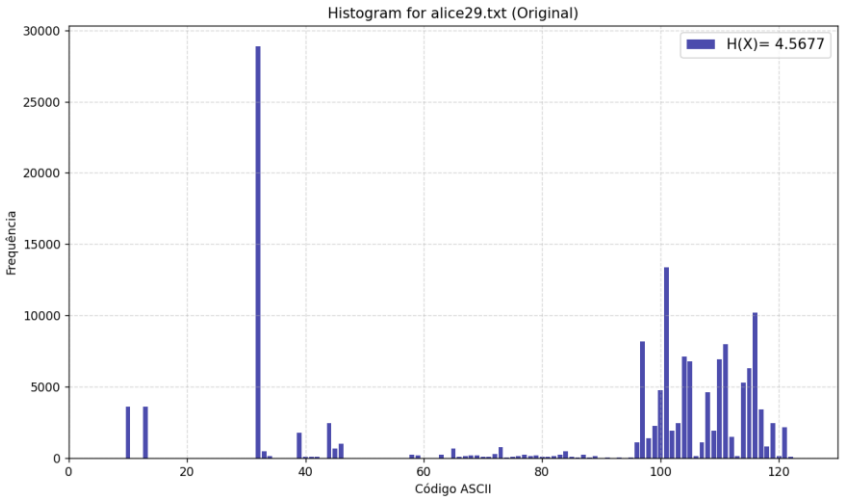
- alice29.txt
  - Entropia do ficheiro original: 4.5677 bits/símbolo
  - Entropia do ficheiro cifrado: 4.8527 bits/símbolo
  - Entropia do ficheiro decifrado: 4.5677 bits/símbolo
- asyoulik.txt
  - Entropia do ficheiro original: 4.8081 bits/símbolo
  - Entropia do ficheiro cifrado: 5.1372 bits/símbolo
  - Entropia do ficheiro decifrado: 4.8081 bits/símbolo

Os valores de entropia revelam que:

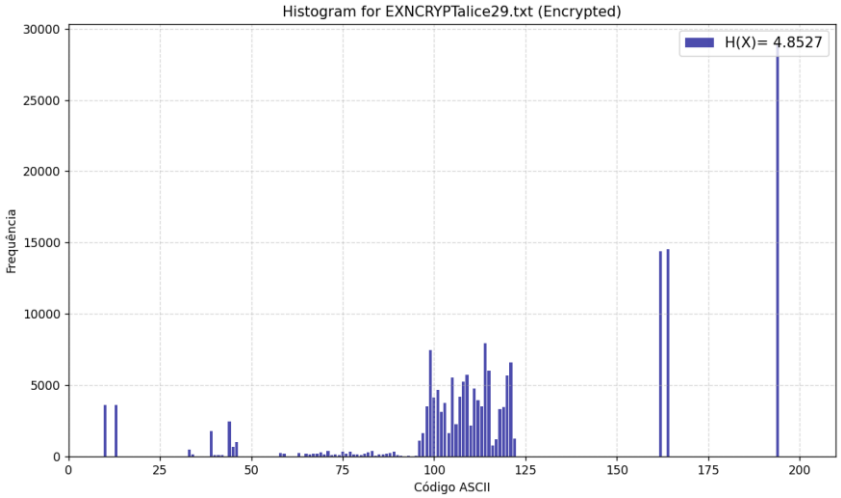
- A entropia nos ficheiros cifrados aumentou em relação aos ficheiros originais. Este aumento é expectável, dado que a cifra introduz mais diversidade nos símbolos utilizados, especialmente devido aos deslocamentos nos caracteres e à substituição aleatória dos espaços por símbolos especiais. Isso torna a distribuição dos caracteres mais uniforme, o que a torna menos previsível e, portanto, aumenta a entropia.
- A entropia dos ficheiros decifrados corresponde, com alta precisão, à dos ficheiros originais. Anteriormente, observámos uma ligeira diferença na entropia do ficheiro asyoulike.txt (uma redução de 0.0005), causada pela substituição incorreta de cinco caracteres do texto original por espaços durante o processo de cifra e decifra. Estes cinco caracteres ('&', '@', '#', '%') faziam parte da lista de símbolos especiais que utilizávamos para substituir aleatoriamente os espaços com o objetivo de disfarçar a estrutura do texto. Como estavam também presentes no conteúdo original, acabaram por ser confundidos com substituições de espaços, erro que identificámos com o apoio da classe *Counter* da biblioteca *collections*. Posteriormente, alterámos os símbolos especiais utilizados para '¤' e '¢', que são muito menos comuns nos textos analisados. Esta mudança eliminou as ambiguidades e permitiu que os ficheiros decifrados preservassem com exatidão a entropia dos originais, garantindo a integridade estatística do conteúdo.

Histogramas:

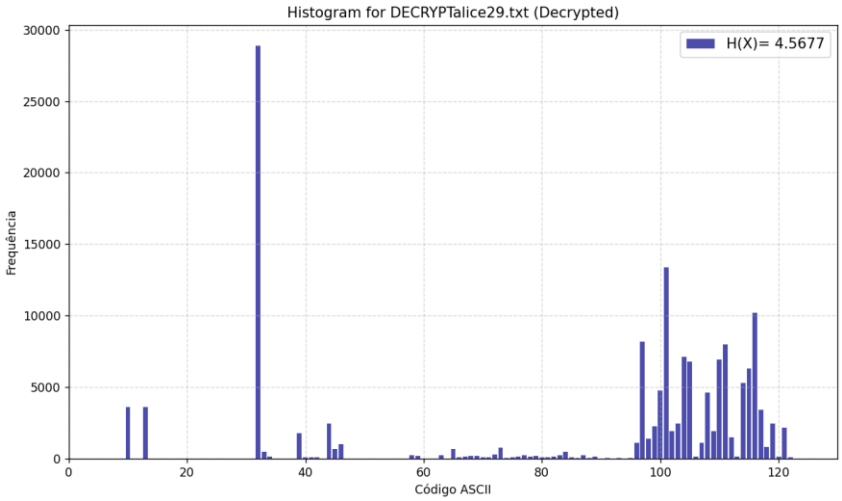
- alice29.txt
  - Original



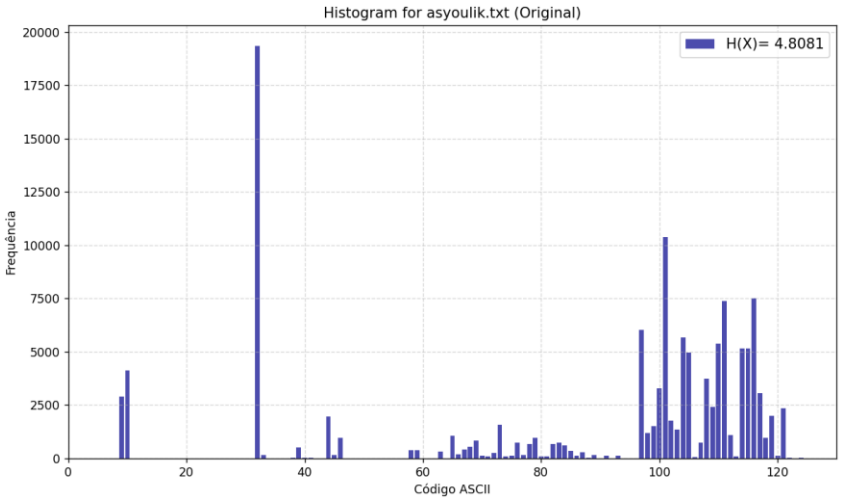
  - Cifrado



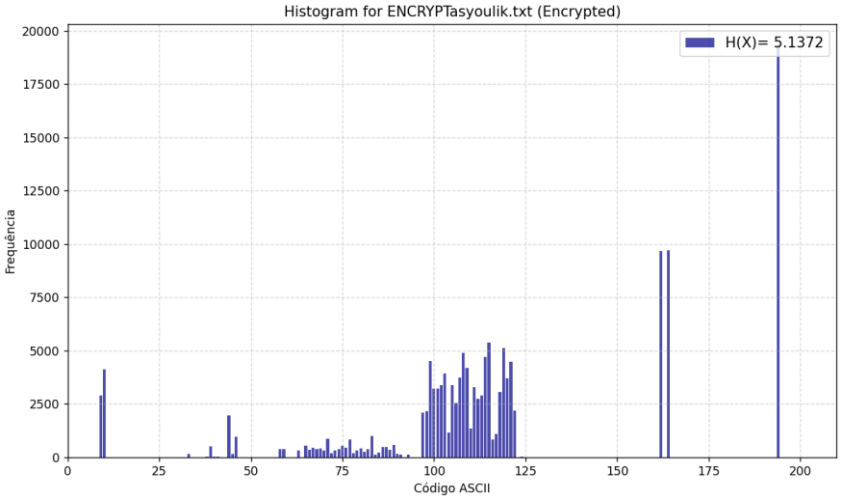
  - Decifrado



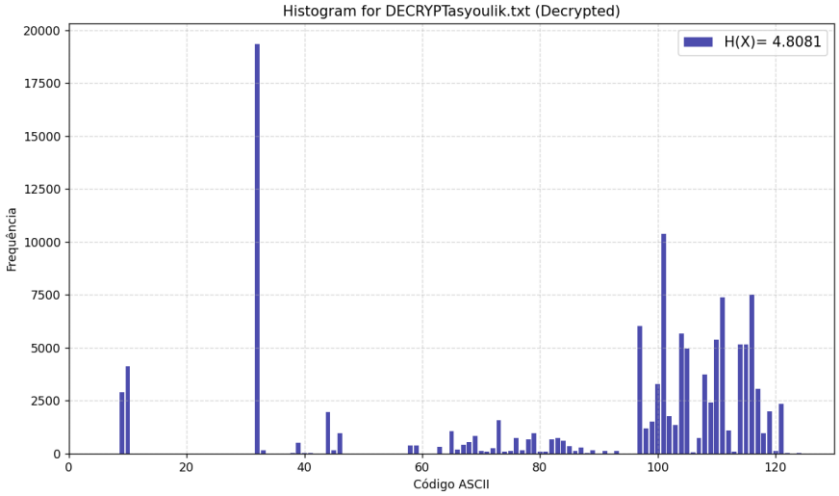
- asyoulik.txt
  - Original



  - Cifrado



  - Decifrado



Comentário: Estes resultados confirmam o correto funcionamento das funções de encriptação e desencriptação. A cifra de Vigenère personalizada preserva a integridade da informação original, enquanto aumenta a entropia do conteúdo cifrado, dificultando uma análise estatística direta.